

Relatório — Sistema de Chat Distribuído

Aluno(s): Darmes Dias e Gabriel Neri
Data: 26/06/2026
Repositório: https://github.com/minhoad/chat_distribuido

1. Introdução e Objetivos

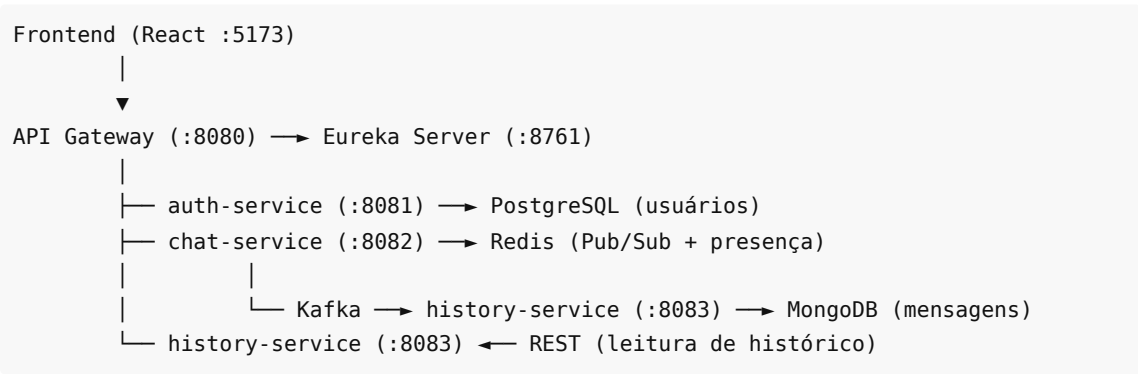
Este relatório descreve a implementação de uma plataforma de comunicação em tempo real com arquitetura distribuída, desenvolvida como trabalho prático de Sistemas Distribuídos. O objetivo principal é permitir que múltiplos usuários troquem mensagens instantâneas de forma confiável, atendendo aos requisitos de **alta disponibilidade**, **comunicação em tempo real** e **escalabilidade horizontal**.

A solução adota **Java 21** com **Spring Boot 3.4** no backend, organizada em Maven multi-módulo, e **React + Vite** no frontend. A infraestrutura de apoio (PostgreSQL, MongoDB, Redis, Kafka e Zookeeper/KRaft) é provisionada via **Docker Compose**.

2. Arquitetura e Decisões de Projeto

2.1 Visão geral

O sistema divide responsabilidades em microsserviços independentes, com descoberta de serviços e ponto de entrada único:



Componente	Tecnologia	Justificativa
Linguagem / runtime	Java 21 + Spring Boot 3.4	Ecossistema maduro, suporte a Virtual Threads
Autenticação	Spring Security + JWT (stateless)	Escalável; token compartilhado entre HTTP e WebSocket
Tempo real	WebSocket + STOMP (SockJS)	Push persistente exigido pelo enunciado
Escrita de mensagens	chat-service	Recebe via STOMP, publica em Redis e Kafka

Leitura de histórico	history-service	REST sobre MongoDB (CQRS leve)
DB relacional	PostgreSQL	Consistência para usuários e credenciais
DB NoSQL	MongoDB	Escrita rápida e schema flexível para mensagens
Comunicação assíncrona	Apache Kafka	Desacopla entrega em tempo real da persistência
Distribuição entre instâncias	Redis Pub/Sub	Roteia mensagens entre réplicas do chat-service
Descoberta / balanceamento	Netflix Eureka + Spring Cloud Gateway	Registro dinâmico e roteamento lb://
Contrato compartilhado	Módulo chat-common	Evita divergência de serialização Kafka entre serviços
Concorrência I/O	Virtual Threads (spring.threads.virtual.enabled=true)	Suporta muitas conexões WebSocket com código síncrono

Kafka e Redis — papéis e o que *não* são

O enunciado não exige Kafka nem Redis; ambos foram escolhidos para demonstrar padrões distribuídos reais, com trade-off de complexidade operacional.

Componente	Papel no projeto	Alternativa mais simples
Kafka	Desacopla a persistência (history-service) da entrega em tempo real (chat-service). Mensagens publicadas no tópico chat-messages sobrevivem a reinícios do consumidor.	Gravar direto no MongoDB a partir do chat-service (acopla serviços e bloqueia o chat se o Mongo estiver lento).
Redis Pub/Sub	Sincroniza mensagens entre réplicas do chat-service via canal chat:global.	Desnecessário com uma única instância do chat.
Redis (presença)	Chaves user:session:* e user:online:* para estado de conexão.	Não é cache de consultas HTTP/DB — esse uso seria opcional (ex.: cachear histórico por alguns segundos).

Com **uma instância** de cada serviço, o ganho prático do Kafka e do Redis Pub/Sub é pequeno na demo, mas a arquitetura fica preparada para escala e falhas parciais — desde que validada empiricamente (seções 4.3 e 4.4).

2.2 Separação escrita vs. leitura

Adotou-se um padrão **CQRS leve**:

- **Escrita:** o cliente envia mensagem via STOMP (`/app/chat.send`); o `chat-service` valida o remetente, publica no Redis (entrega imediata) e no Kafka (persistência assíncrona).
- **Leitura:** o frontend consulta `GET /api/history/conversation/{userId}/{peerId}` ou `GET /api/history/recipient/{groupId}` ao abrir uma conversa.

Essa separação evita que operações de banco bloqueiem a entrega em tempo real.

2.3 Fluxo de uma mensagem privada (1:1)

1. Usuário A envia payload JSON via STOMP para `/app/chat.send`.
2. O interceptor WebSocket valida o JWT no `CONNECT` e associa a sessão ao `userId`.
3. O `ChatController` confere se `senderId` coincide com o usuário autenticado.
4. A mensagem é publicada no canal Redis `chat:global`.
5. Cada instância do `chat-service` inscrita recebe o evento e entrega via `convertAndSendToUser` para remetente e destinatário (`/user/queue/messages`).
6. Em paralelo, a mensagem é enviada ao tópico Kafka `chat-messages`.
7. O `history-service` consome e persiste no MongoDB.

2.4 Mensagens em grupo (1:N)

Grupos são identificados por `recipientId` (ex.: `sala-geral`, `projeto-sd`) com `type: GROUP`. A entrega usa broadcast STOMP em `/topic/group.{groupId}`. O histórico de grupo é recuperado por `GET /api/history/recipient/{groupId}`. Os grupos são **fixos no frontend** — não há CRUD de salas no backend.

2.5 API Gateway — desenvolvimento vs apresentação

A arquitetura alvo prevê **entrada única** pelo API Gateway (`:8080`), com roteamento via Eureka (`lb://auth-service`, `lb://chat-service`, etc.). Na prática, existem dois modos de operação do frontend:

Modo	Comando	Roteamento REST	Roteamento WebSocket	Uso
Desenvolvimento	<code>npm run dev</code>	Vite → microserviços (<code>:8081</code> , <code>:8083</code>)	Vite → <code>:8082/ws</code>	Codificação diária; um salto de proxy
Apresentação / demo	<code>npm run dev:gateway</code>	Vite → Gateway <code>:8080</code>	Browser → Gateway <code>:8080/ws</code> (sem proxy Vite)	Demo da arquitetura distribuída
Teste de carga REST	<code>load-test/run_load_test.ps1</code>	Cliente → Gateway <code>:8080</code>	—	Auth + histórico via entrada única

WebSocket no modo `dev:gateway` : o SockJS (STOMP) realiza várias requisições HTTP antes do upgrade WebSocket. Um proxy duplo (Vite → Gateway → chat) provoca `ECONNRESET` e reconexões periódicas. A correção adotada separa os caminhos:

- **REST** (login, usuários, histórico): browser → Vite `:5173` → proxy → Gateway `:8080`.

- **WebSocket:** browser conecta **diretamente** em `localhost:8080/ws` (`VITE_WS_BASE` em `.env.gateway`), sem passar pelo proxy do Vite; o Gateway roteia ao `chat-service` via Eureka.

Isso demonstra entrada única pelo Gateway sem sacrificar estabilidade do STOMP. Fallback documentado: `VITE_WS_BASE=http://localhost:8082/ws` (REST via Gateway, WS direto no chat).

2.6 Portabilidade e reprodutibilidade

Para rodar em máquinas distintas (requisito prático da entrega), foram aplicadas:

Melhoria	Detalhe
Volumes Docker nomeados	<code>postgres_data</code> , <code>mongo_data</code> — sem bind mounts com caminho absoluto (<code>D:/...</code>)
<code>restart: unless-stopped</code>	Containers de infra sobem após reinício do Docker Desktop
Maven Wrapper	<code>mvnw / mvnw.cmd</code> — Maven fixo sem instalação global
Makefile + <code>run.sh</code>	Atalhos: <code>infra</code> , <code>build</code> , <code>test</code> , <code>eureka</code> , <code>backends</code> , <code>front-gateway</code> , <code>load-test</code> , <code>health</code> , <code>chaos-test</code>
Scripts Windows	<code>start-eureka.ps1</code> , <code>start-backends.ps1</code> (sem <code>JAVA_HOME</code> hardcoded)

Pré-requisitos que o projeto não instala: Java 21+, Docker Engine/Desktop, Node.js 18+. No Windows, `./run.sh backends` delega ao PowerShell; no Linux/macOS, abrir terminais separados (`make auth` , `make chat` , etc.) ou usar `run.sh` .

Ordem de subida validada: `docker compose up -d` → aguardar 30 s (Kafka) → Eureka (10 s) → `backends` → frontend. Se o PostgreSQL estiver parado, o `auth-service` falha com `Connection refused :5432` até o container voltar; o pool Hikari pode exigir **reinício manual** do auth após queda prolongada do banco.

2.7 Limitações arquiteturais reconhecidas

- **Alta disponibilidade:** infraestrutura preparada (Eureka, Gateway, Redis), com **testes de caos parciais** na camada Docker (seção 4.4). Não há failover automatizado de instâncias Java nem medição de RTO/RPO.
- **Escalabilidade horizontal:** Redis Pub/Sub implementado, porém **não demonstrado** com duas instâncias do `chat-service` simultâneas.
- **Grupos:** sem gestão dinâmica de membros ou permissões.
- **Zookeeper no compose:** presente na infra, mas o Kafka opera em modo KRaft — o Zookeeper é redundante operacionalmente, mantido por decisão de não alterar a stack na entrega.

3. Implementação

3.1 Serviço de Autenticação (`auth-service`)

Responsável pelo ciclo de vida do usuário:

Endpoint	Descrição
----------	-----------

POST /api/auth/register	Cadastro com validação (senha 6-100 caracteres)
POST /api/auth/login	Autenticação; retorna JWT
GET /api/auth/users	Lista usuários para seleção de conversas

Senhas armazenadas com **BCrypt**. Erros expostos em formato padronizado (`ApiError`) com mensagens amigáveis (ex.: senha curta → HTTP 400 com texto explicativo).

3.2 Serviço de Chat (`chat-service`)

- Endpoint WebSocket: `/ws` (SockJS + STOMP).
- Destinos: `/app/chat.send` (entrada), `/user/queue/messages` (privado), `/topic/group.*` (grupo).
- Autenticação STOMP via header `Authorization: Bearer {token}` no `CONNECT`.
- `PresenceService` registra sessões no Redis (`user:session:*` , `user:online:*`).

3.3 Serviço de Histórico (`history-service`)

- Consumer Kafka persiste `ChatMessage` como `ChatMessageDocument` no MongoDB.
- Consultas: conversa bidirecional 1:1 (`findConversation`) e mensagens por destinatário/grupo.

Correção relevante: inicialmente, producer e consumer usavam classes diferentes no Kafka, o que impedia a persistência. Foi criado o módulo `chat-common` e desabilitado o header de tipo na serialização, restaurando o fluxo de gravação.

3.4 Frontend (`frontend/`)

Interface responsiva (CSS com `@media (max-width: 768px)`) contendo:

- Tela de login/registro.
- Sidebar com lista de usuários (atualizada a cada 15 s via HTTP) e grupos pré-definidos.
- Área de mensagens com envio por Enter ou botão.
- Indicador de conexão WebSocket (`conectado` / `desconectado`).
- Exibição de erros de chat e de histórico.

O chat em tempo real **não utiliza polling HTTP** — mensagens novas chegam exclusivamente via STOMP. O histórico é carregado uma vez ao selecionar conversa ou grupo.

4. Testes e Avaliação

4.1 Testes automatizados

Execução: `.\mvnw.cmd test` (Java 21). **9 testes, 0 falhas** (verificado em 26/06/2026).

Módulo	Classe	Tipo	O que cobre
auth-service	AuthServiceTest	Unitário	Login válido/inválido, registro
auth-service	JwtServiceTest	Unitário	Geração e validação de token
auth-service	AuthIntegrationTest	Integração	Fluxo register → login via MockMvc (H2 em memória)

history-service	HistoryServiceTest	Unitário	Persistência e ordenação de conversa
-----------------	--------------------	----------	--------------------------------------

Lacunas honestas:

- **chat-service não possui testes automatizados** (WebSocket, Redis, Kafka).
- **Não há teste de integração ponta-a-ponta** (login → envio STOMP → verificação no MongoDB).
- Integração do auth usa **H2**, não PostgreSQL real (sem Testcontainers).

4.2 Teste E2E automatizado (auth + STOMP)

Script `load-test/e2e_auth_stomp.mjs` (Node + `@stomp/stompjs` + SockJS — mesmas libs do frontend):

Etapa	O que valida
1	POST <code>/api/auth/register</code> via Gateway (2 usuários)
2-3	STOMP CONNECT com JWT em <code>ws_url</code> (default <code>:8080/ws</code>)
4	Envio GROUP → recebimento em <code>/topic/group.sala-geral</code>
5	Best-effort: GET <code>/api/history/recipient/sala-geral</code> após 3 s

Execução: `cd load-test && npm install && npm run e2e` (stack completa UP).

Resultado observado (27/06/2026, Gateway + serviços quentes):

Métrica	Valor
Auth (2 usuários)	823 ms
STOMP connect receiver	111 ms
STOMP connect sender	21 ms
Entrega mensagem (STOMP)	125 ms
Histórico no Mongo	Confirmado (132 ms após 3 s de espera)
Integração auth → chat	PASS

Escopo honesto: cobre o fluxo exigido pelo enunciado (“autenticar e enviar mensagem”), mas com **2 usuários** e **1 mensagem**, não 10 concorrentes. Não testa chat 1:1 privado (usa grupo `sala-geral`).

4.3 Testes manuais (estudos de caso funcionais)

Cenário	Procedimento	Resultado observado
Registro e login	Dois usuários em abas distintas	Tokens distintos; lista de usuários atualizada
Chat 1:1	Selecionar peer, enviar mensagens	Entrega em tempo real para ambos; histórico ao reabrir conversa

Chat 1:N	Enviar na "Sala Geral"	Todos os conectados ao tópico recebem; histórico via /recipient/sala-geral
Validação de senha	Registrar com senha < 6 caracteres	HTTP 400 com mensagem amigável
Infra indisponível	Kafka/MongoDB parados	Chat em tempo real via Redis funciona; histórico falha ou retorna vazio

4.4 Teste de concorrência/carga (REST)

Script `load-test/run_load_test.ps1` : **10 usuários** via Gateway — register/login + GET /api/history/recipient/sala-geral .

Modo	Comando	O que mede
Sequencial	<code>.\run_load_test.ps1</code>	10 fluxos REST em fila
Paralelo	<code>.\run_load_test.ps1 -Parallel</code>	10 fluxos REST simultâneos (Start-Job no PS 5; ForEach-Object -Parallel no PS 7+)

Resultados observados (27/06/2026, Gateway :8080 UP):

Modo	Tokens	Tempo total (parede)	Latência média/usuário
Sequencial	10/10	~1062 ms	~104 ms
Paralelo	10/10	~1898 ms	~223 ms

O modo paralelo **dispara 10 register/login ao mesmo tempo** no mesmo PostgreSQL/auth-service — comprova que o serviço aguenta requisições concorrentes, mas o tempo de parede pode ser **maior** que o sequencial (contenção no banco + overhead de jobs).

Limitações (permanecem):

Aspecto	Situação
Envio de mensagens chat	Não — só REST; use <code>npm run e2e</code> para STOMP
10 WebSockets simultâneos	Não — JMeter/Gatling seria o complemento
Balanceamento multi-instância	Não — uma instância de cada serviço
Escalabilidade horizontal	Redis implementado, não comprovado com 2x chat-service

4.5 Testes de disponibilidade (chaos / resiliência)

Complementam os testes Maven com cenários de falha na **infraestrutura Docker**. Scripts em `chaos-test/` (PowerShell, ASCII-only para evitar erros de encoding no Windows).

Script	Falha simulada	Comportamento esperado	Resultado observado (26/06/2026)

scenario_01_network_postgres.ps1	docker stop chat-postgres	Auth indisponível	Auth falhou (timeout) — esperado
	Restauração	Auth recupera após postgres healthy	Auth voltou sem reinício manual nesta execução; o script alerta que o Hikari pode exigir restart do auth-service
scenario_02_network_redis.ps1	docker stop chat-redis	Auth OK; chat tempo real degradado	Auth :8081 e via Gateway :8080 OK ; degradação do chat não verificada no browser
scenario_03_latency_pause.ps1	docker pause chat-kafka (20 s)	Persistência assíncrona atrasa	History respondeu lendo Mongo (~5 ms); fila retoma após unpause — coerente
scenario_04_memory_pressure.ps1	docker update -memory 32m no Redis	OOM ou degradação	Container permaneceu running, sem OOM
check_health.ps1	Diagnóstico	14 checks (Docker, portas, Eureka, auth, gateway)	14/14 OK antes e após suite
run_all_chaos.ps1	Sequência 1-4	Exit code 0	Suite completa passou (-SkipMemory ou com memória)

Escopo dos testes de caos:

- Validam **degradação e recuperação da infra** (DB, fila, cache), não failover de réplicas Java.
- Cenário 2 não abriu conexões WebSocket durante a queda do Redis.
- Efeitos colaterais entre cenários são possíveis (ex.: timeout no Gateway após queda do Postgres se o pool ainda estiver inválido).

Execução: `cd chaos-test && .\check_health.ps1` ou `make chaos-test` / `make health` na raiz.

4.6 Matriz de conformidade com o enunciado do TP

Requisito	Atendido	Observação
≥ 2 microsserviços	Sim	auth, chat, history (+ gateway, eureka)
WebSocket tempo real	Sim	STOMP/SockJS

DB usuários + mensagens	Sim	PostgreSQL + MongoDB
Frontend responsivo	Sim	React com layout adaptável
Mensagens 1:1 e 1:N	Sim	PRIVATE e GROUP
Persistência de histórico	Sim	Kafka → MongoDB (após correção do contrato)
Testes unitários	Parcial	auth e history; chat sem testes
Testes de integração	Parcial	E2E auth+STOMP (e2e_auth_stomp.mjs); auth MockMvc isolado; sem front automatizado
Teste carga 10 usuários	Parcial	10/10 tokens REST paralelos ; E2E STOMP com 2 users; sem 10 WS concorrentes
Disponibilidade / resiliência	Parcial	scripts chaos-test/ validados (4 cenários infra); sem failover de serviços Java
Alta disponibilidade comprovada	Parcial	degradação/recuperação de Postgres, Redis, Kafka testada; RTO não medido
Escalabilidade horizontal comprovada	Não testado	Redis Pub/Sub implementado, sem demo multi-instância

5. Conclusões

1. Foi construída uma arquitetura distribuída funcional, com separação clara entre autenticação, mensageria em tempo real e persistência de histórico, utilizando PostgreSQL, MongoDB, Redis, Kafka, Eureka e API Gateway.
2. A escolha de **Virtual Threads** e **STOMP sobre WebSocket** equilibra simplicidade de implementação e capacidade de atender muitas conexões simultâneas.
3. O módulo **chat-common** e a correção da serialização Kafka foram decisivas para que a persistência funcionasse de fato — antes disso, mensagens apareciam em tempo real, mas o histórico permanecia vazio.
4. Os **testes automatizados cobrem a lógica central de autenticação e histórico**, porém o **chat-service** e o **fluxo integrado completo carecem de cobertura**.
5. O **teste de carga REST paralelo** obteve 10/10 tokens via Gateway (~1,9 s parede, contenção no auth). O **E2E auth+STOMP** validou entrega em ~125 ms e persistência no histórico — ainda **não** equivale a 10 usuários trocando mensagens via WebSocket simultaneamente.
6. Os **testes de caos** (chaos-test/) cobrem indisponibilidade de Postgres, Redis, pausa do Kafka e pressão de memória, com resultados alinhados ao desenho (auth falha sem DB; history lê Mongo com Kafka pausado). São **parciais**: não medem chat WebSocket durante falhas nem failover de microsserviços Java.
7. A **portabilidade** foi reforçada (volumes nomeados, Makefile , run.sh , políticas de restart). O modo **dev:gateway** demonstra arquitetura com entrada única, com WebSocket direto ao

Gateway para contornar limitação do SockJS com proxy duplo.

Referências

1. Spring Boot 3.4 Documentation — <https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. Spring WebSocket/STOMP — <https://docs.spring.io/spring-framework/reference/web/websocket.html>
3. Apache Kafka Documentation — <https://kafka.apache.org/documentation/>
4. Redis Pub/Sub — <https://redis.io/docs/interact/pubsub/>
5. Spring Cloud Netflix Eureka — <https://spring.io/projects/spring-cloud-netflix>